

All-Weather Moving Object Detection

Using Radar-Centric Multimodal Deep Learning

Detailed Project Description - based on the Computer Vision Project brief and the Radar Detection Pipeline Document.

1. Project Goal

Design and implement a deep-learning system that detects moving objects using radar as the primary sensing modality, optionally combined with LiDAR and RGB camera data. Unlike conventional vision systems that rely heavily on cameras or LiDAR, the project emphasizes radar-based perception, which is robust under adverse weather conditions such as rain, fog, and snow. The system must maintain high detection accuracy when other sensors degrade.

2. Datasets

Primary (recommended):

- K-Radar - synchronized 4D radar, RGB camera, and LiDAR with 3D bounding boxes across clear, rain, and fog scenarios. <https://github.com/kaist-avelab/K-Radar>

Alternative datasets (optional):

- RADIATE - radar + camera + LiDAR, adverse weather
- nuScenes - multimodal autonomous driving dataset
- Oxford Radar RobotCar Dataset
- Any similar multimodal dataset with radar support

3. Main Objectives

- Understand radar-based perception and its advantages over camera/LiDAR in harsh environments.
- Develop a deep-learning model for moving object detection using radar data.
- Implement multimodal fusion (encouraged): Radar+Camera, Radar+LiDAR, or Radar+Camera+LiDAR.
- Compare performance across modalities (radar-only vs. multimodal) and across weather splits.
- Handle real-world challenges: noise in radar signals, sparse / low-resolution data, and temporal consistency for motion modeling.

4. System Architecture

The pipeline takes raw multi-sensor frames, preprocesses radar (CFAR filtering and Bird's-Eye-View projection), runs separate backbones for each modality, fuses them via cross-attention, and emits bounding boxes through a YOLOv8 detection head. Training uses PyTorch Lightning and tracking is done with Weights & Biases.

4.1 Pipeline Flow



5. Pipeline Components

Stage	Module / Tool	Output
Data Loading	dataset.py -> KRadarDataset	Synchronized radar/camera/labels
Radar Preproc.	preprocessing.py -> RadarPreprocessor	2-ch BEV (intensity+velocity)
Camera Preproc.	preprocessing.py -> CameraPreprocessor	Normalized (3,640,640) RGB
Radar Backbone	models/radar_yolo.py -> RadarYOLO	Radar feature map F_R
Camera Backbone	timm ResNet-50	Camera feature map F_C
Fusion	models/fusion.py -> CrossModalAttention	Fused feature map F_fused
Detection Head	YOLOv8 head	Boxes + class + confidence
Training	train.py (PyTorch Lightning)	Best checkpoint (.ckpt)
Evaluation	evaluate.py (COCOeval)	mAP@50, mAP@50:95, weather mAP
Visualization	visualize.py	Annotated BEV maps, charts
Tracking	Weights & Biases	Metrics + visuals logged

6. Environment Setup

Key dependencies (requirements.txt):

```
torch>=2.1.0          torchvision>=0.16.0
pytorch-lightning>=2.1.0
ultralytics>=8.0.0     # YOLOv8
open3d>=0.17.0         # Point cloud visualization
numpy>=1.24.0          scipy>=1.11.0
opencv-python>=4.8.0   matplotlib>=3.7.0
wandb>=0.16.0          # Experiment tracking
einops>=0.7.0          # Attention tensor ops
timm>=0.9.0            # Vision backbones
pyarrow>=13.0.0        h5py>=3.9.0
```

Installation:

```
conda create -n radar_det python=3.10 -y
conda activate radar_det
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
pip install -r requirements.txt
git clone https://github.com/kaist-avelab/K-Radar.git
cd K-Radar && pip install -e .
```

7. Radar Preprocessing (CFAR + BEV)

Radar signals contain heavy clutter and noise. Constant False Alarm Rate (CFAR) detection is applied to extract genuine target reflections, then the polar (range-azimuth) map is projected into a Cartesian Bird's-Eye-View. The output is a 2-channel BEV tensor: intensity + Doppler velocity, normalized.

- Input: 4D radar tensor (R, Az, El, Doppler).
- Power spectrum -> CFAR mask -> elevation collapse -> BEV grid.
- Doppler channel preserves motion information for moving-object discrimination.

8. Fusion Strategy Comparison

Strategy	Pros	Cons
Early Fusion (concat before backbone)	Simple to implement	Camera artifacts can hurt radar
Late Fusion (merge at detection head)	Modular, easy to ablate	No cross-modal interaction
Cross-Attention (transformer over feats)	Best performance, SOTA	More compute, complex to tune

Recommended: Bidirectional Cross-Modal Attention. Radar features attend to camera context and vice versa, enabling each modality to refine the other before detection.

9. Training Setup

- Framework: PyTorch Lightning with mixed precision (fp16) and gradient clipping.
- Optimizer: AdamW with weight decay 1e-4.
- Scheduler: CosineAnnealingLR over the full epoch budget.
- Logging: every 10 steps; checkpoint best validation mAP.
- Procedure: train the radar-only baseline first, then the fusion model.

10. Evaluation Metrics

Metric	Tool / Definition	Why It Matters
mAP@50	COCO API - IoU 0.50	Standard detection benchmark
mAP@50:95	COCO API - avg IoU 0.50-0.95	Stricter, rewards localization
Precision	TP / (TP + FP)	Correctness of detections
Recall	TP / (TP + FN)	Completeness of detections
Weather mAP	mAP per clear/rain/fog split	CORE - shows robustness
Latency (ms)	Inference time per frame on GPU	Real-time feasibility

11. Visualization & Experiment Tracking

- Draw bounding boxes on radar BEV maps (color-coded per class) and save annotated frames.
- Bar chart of mAP@50 per weather condition (clear / rain / fog).
- Weights & Biases: log train/val loss, mAPs, weather splits, hyperparameters, BEV images.

12. Expected Deliverables

- Fully working implementation (clean repo).
- Trained model checkpoints with reproducible results.
- Evaluation metrics: mAP, precision, recall, per-weather mAP.
- Visualization results - detections on radar / fused data.
- Comparison across weather conditions and across modalities.
- Final report covering methodology and analysis.

13. Recommended Repo Structure

```
radar-detection/  
  configs/           # YAML configs per experiment  
  dataset/           # KRadarLayout + custom loaders  
  preprocessing/     # CFAR, BEV projection, image norm  
  models/  
    radar_yolo.py    # YOLOv8 adapted for 2-ch radar BEV  
    fusion.py        # CrossModalAttention  
  evaluate/          # COCOeval + per-weather metrics  
  visualize/         # BEV drawing, weather charts  
  train.py           # PyTorch Lightning entrypoint  
  evaluate.py        # Evaluation entrypoint  
  requirements.txt  
  README.md
```

14. Suggested Execution Order

1. Set up conda env and clone K-Radar.
2. Implement dataset.py + preprocessing.py; verify a few BEV maps render correctly.

- 3. Train radar-only YOLOv8 baseline; log mAP to W&B.
- 4. Add camera ResNet-50 branch + cross-attention fusion; train and compare.
- 5. Run evaluation with per-weather splits.
- 6. (Optional) Add LiDAR branch as a third modality.
- 7. Write the report and clean up the repo.

15. Key Reminder

Always run the radar-only baseline FIRST and log its mAP before training the fusion model. The delta between baseline and fusion mAP - especially across weather conditions - is the headline result of the project.